Applied Deep Learning

Chapter 6: Recurrent NNs

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Winter 2026

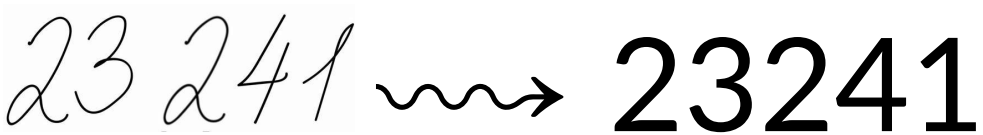
Computing Loss: *Motivating Example*

Let's consider a simple example: *we have an image that includes a sequence of handwritten digits, e.g.,*

- *The sequence includes five digits*
- *Each digit is either 1, 2, 3, or 4*

Our task is to recognize this sequence, i.e., return the five digits in correct order

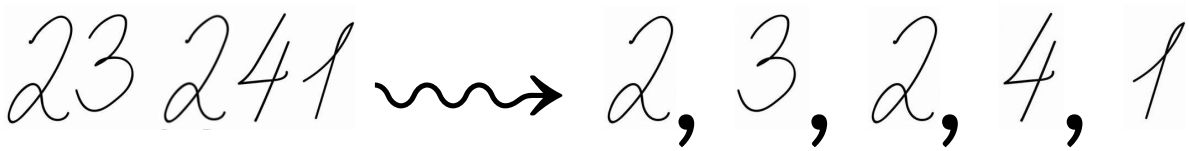
- *This is a classification task*
- *How can we do it?*



Computing Loss: *Motivating Example*

To use a CNN, we need to specify our input size

- *We segment an input image into a sequence of five images*
  - ↳ *These images are all as large as CNN's input size*

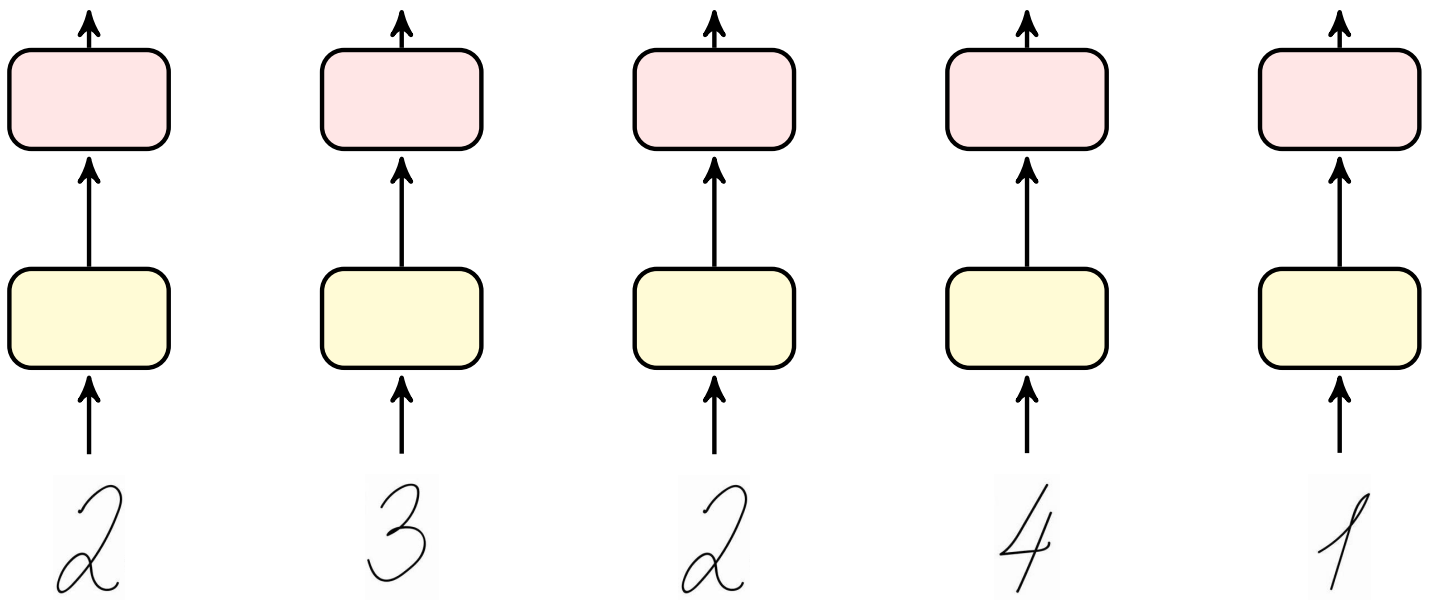


- *We label each image with its label, e.g.,* *is labeled as 2*
- *To compute loss, we compare each output with its corresponding label*

Computing Loss: *Motivating Example*

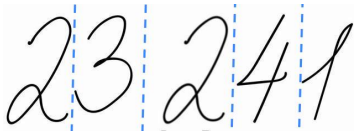
$$\mathbf{v}[1] = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{v}[2] = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \mathbf{v}[3] = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \mathbf{v}[4] = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \mathbf{v}[5] = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} y_1[1] \\ y_2[1] \\ y_3[1] \\ y_4[1] \end{bmatrix} = \mathbf{y}[1] \quad \mathbf{y}[2] \quad \mathbf{y}[3] \quad \mathbf{y}[4] \quad \mathbf{y}[5]$$



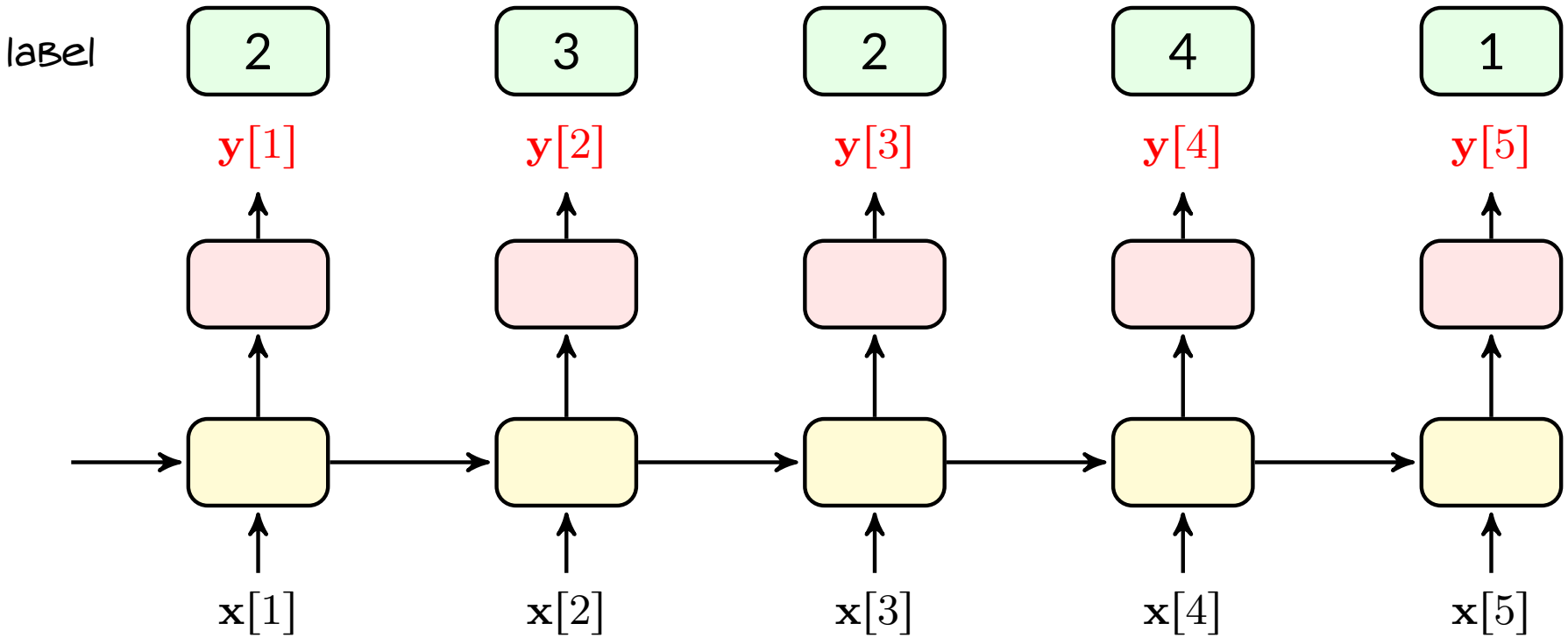
Computing Loss: *One-to-One Correspondence*

If we are extremely lucky; then, our segmentation looks like this



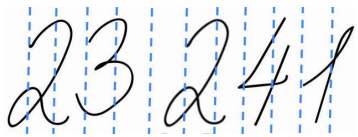
$$\rightsquigarrow \mathbf{x}[1], \mathbf{x}[2], \mathbf{x}[3], \mathbf{x}[4], \mathbf{x}[5]$$

and we have a label for each time step



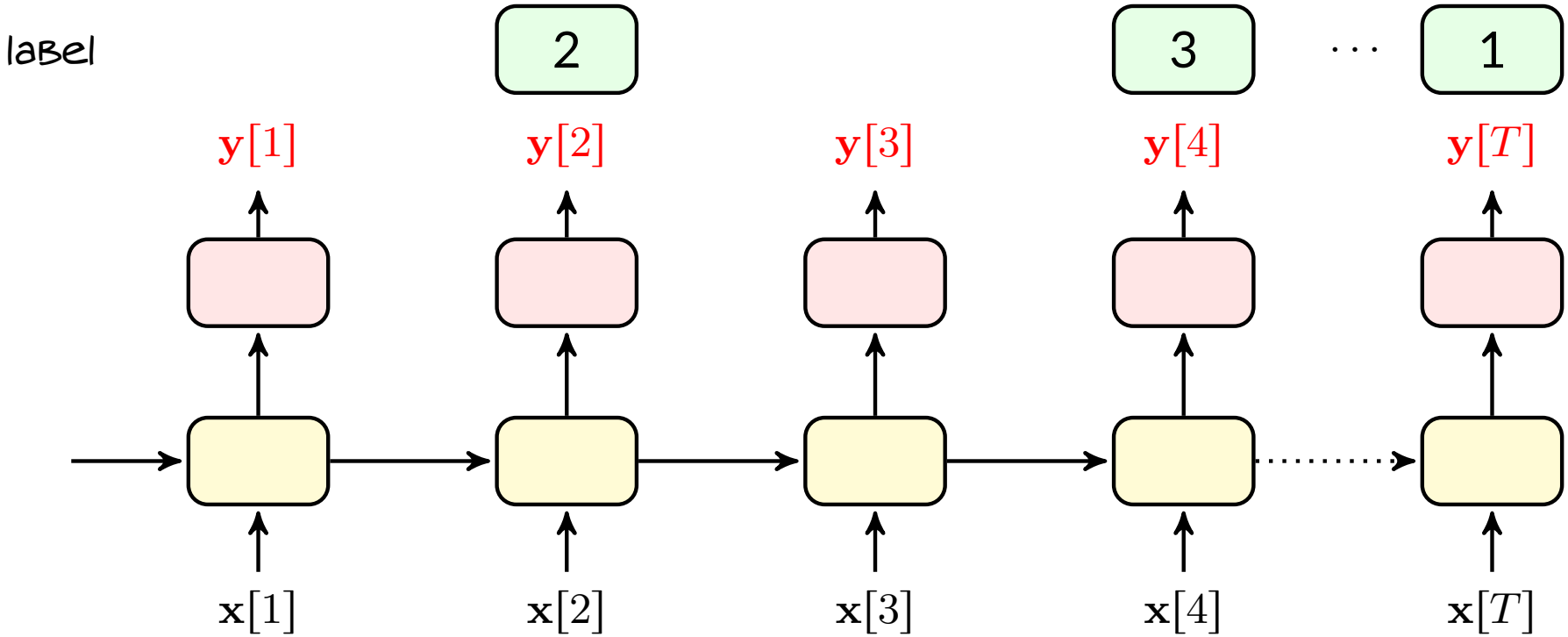
Computing Loss: *One-to-One Correspondence*

But, that's too good to happen! Usually we have



$$\rightsquigarrow \mathbf{x}[1], \mathbf{x}[2], \ldots, \mathbf{x}[T]$$

and we have a label in some time steps



Computing Loss: *One-to-One Correspondence*

The key challenge in computing the loss is that *we do not have necessarily one-to-one correspondence with sequence data*

Correspondence Problem

With sequence data, we could have a data-sequence of length T that is labeled by a sequence of size $K < T$ where

no time index is specified for any label in the K-long label sequence

Correspondence problem exists pretty much in all practical sequence data

- *In speech recognition, multiple time frames correspond to a single word*
- *In text recognition, multiple image frames correspond to a single letter*
- . . .

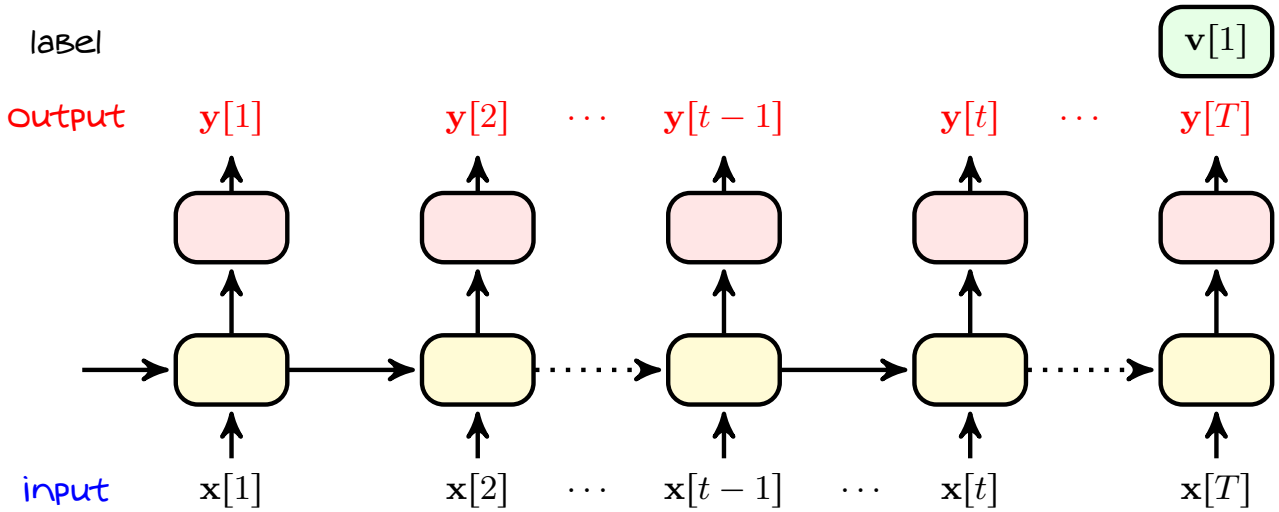
Case I: *Known Segments*

In some problems, *we have only one label for the whole sequence, i.e.,* $K = 1$

↳ *It corresponds to many-to-one type of problems*

↳ *This can be that we have really only one label, e.g., content classification*

↳ *It can be that we know the time index* t *at which each label is assigned*



Case I: *Defining Loss for Dumb NN*

A naive approach to define loss is *to set it be the loss between last output and label*, i.e.,

$$\hat{R} = \mathcal{L}\left(\mathbf{y}[T], \mathbf{v}[1]\right)$$

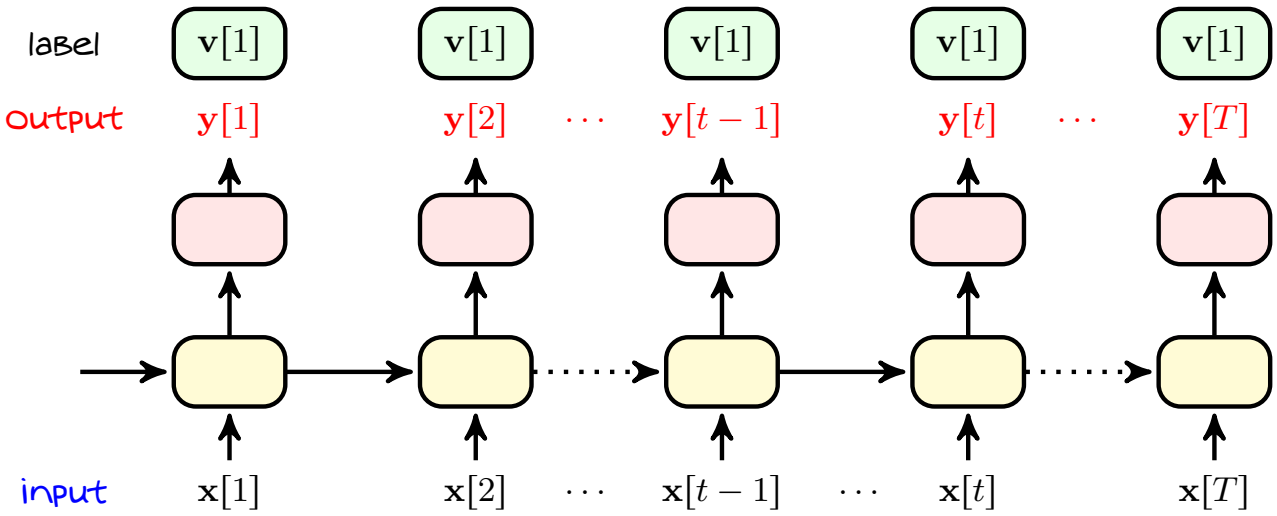
With this loss, *gradient with respect to particular output* $\mathbf{y}[t]$ *is*

$$\nabla_{\mathbf{y}[t]} \hat{R} = \begin{cases} \nabla_{\mathbf{y}[T]} \mathcal{L}\left(\mathbf{y}[T], \mathbf{v}[1]\right) & t = T \\ 0 & t \neq T \end{cases}$$

+ *But does it make sense to ignore all other outputs?*
- Not at all! *We are training a dumb NN that can respond only when it's over with the whole sequence!*

Case I: *Loss for Smarter Training*

An extremely smart NN is the one who knows the label before the input speaks!



For this NN, the loss is

$$\hat{R} = \sum_{t=1}^{T} \mathcal{L}\left(\mathbf{y}[t], \mathbf{v}[1]\right)$$

But, we should be careful! We should not expect NN to know everything from potentially irrelevant input!

Case I: *Defining Proper Loss*

A realistic approach is *to define the loss via a weighted sum*, i.e.,

$$\hat{R} = \sum_{t=1}^{T} w_t \mathcal{L}\left(\mathbf{y}[t], \mathbf{v}[1]\right)$$

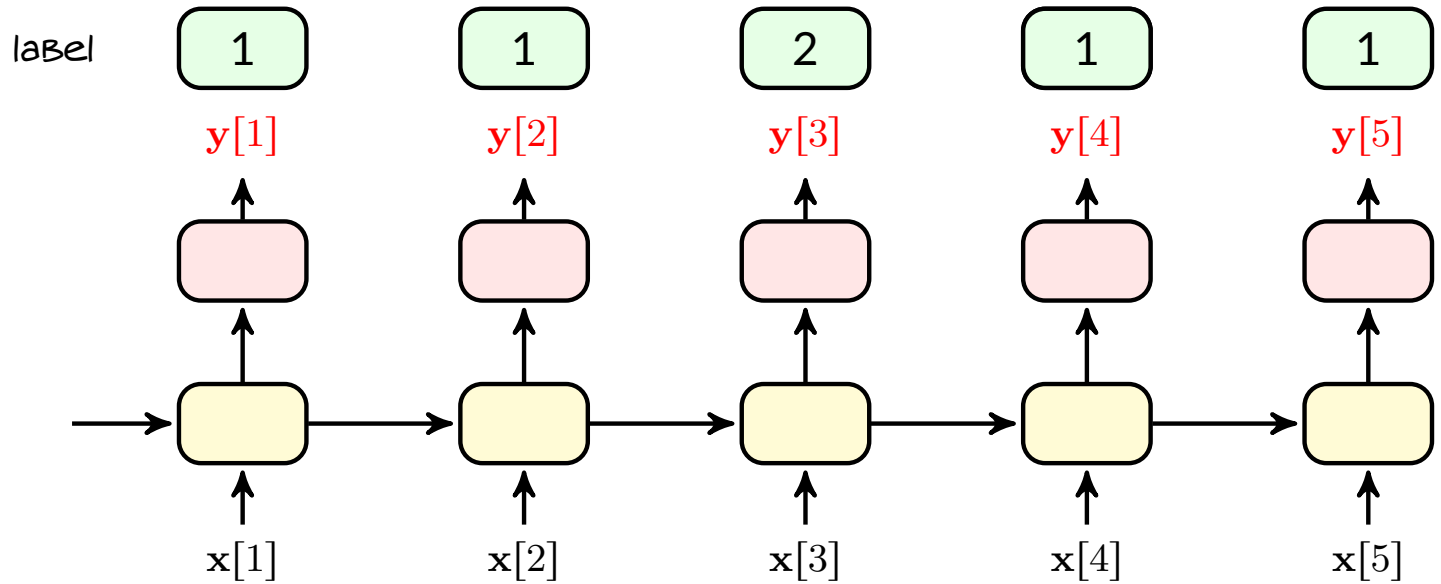
where w_t *is the weight at time* t

- *initially* w_t *is small*
- *it gradually increases up to its maximum* w_T

Case II: *Unknown Segments*

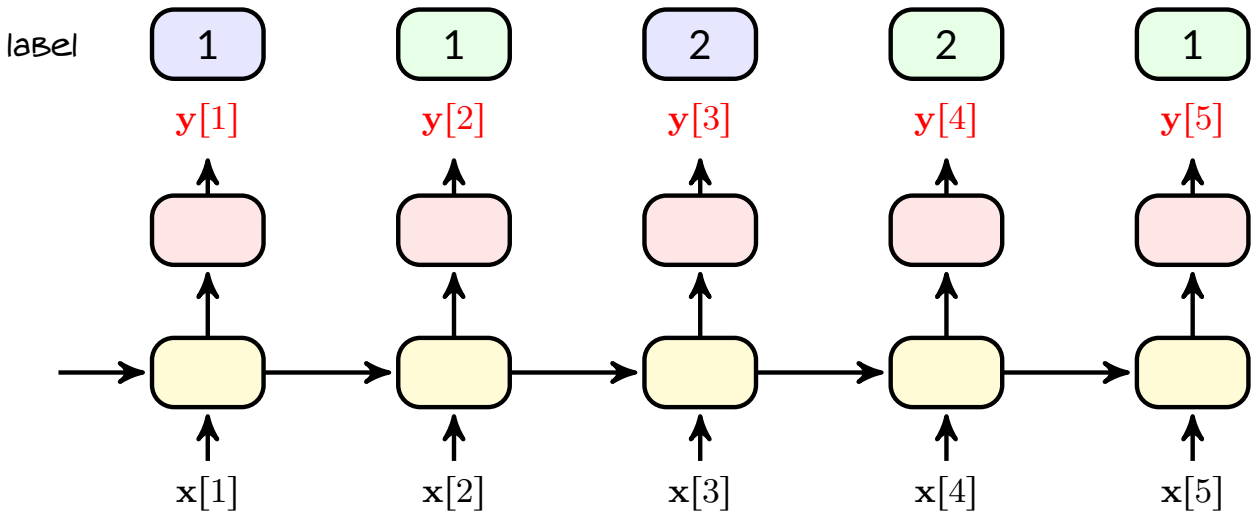
Assume we have image 121 *that is divided into a sequence of five pixel vectors*

- *Since it is a training data, it is labeled as 121*



Case II: *Genie-Defined Loss*

Assume a genie has told us end of each segment

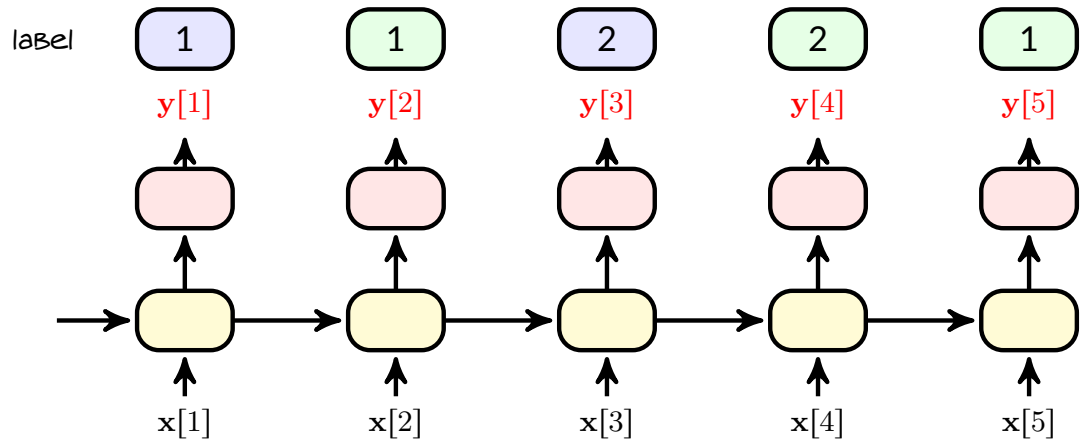


We can fill the empty labels with repetition, and then define the loss as

$$\hat{R} = \sum_{k=1}^{K} \sum_{t=i_{k-1}+1}^{i_k} w_t \mathcal{L}\left(\mathbf{y}[t], \mathbf{v}[k]\right)$$

where i_k *is where label* $\mathbf{v}_k$ *ends, e.g., in above diagram* $i_1 = 2$

Case II: *Defining Loss*



We don't have the genie: we could assume that i_k is something to learn!

$$\hat{R}(\mathbf{i}) = \sum_{k=1}^{K} \sum_{t=i_{k-1}+1}^{i_k} w_t \mathcal{L}(\mathbf{y}[t], \mathbf{v}[k])$$

where $\mathbf{i} = [0, i_1 \ldots, i_K]$ is something we need to learn

Case II: *Optimal Segmentation*

+ *How could we learn* $\mathbf{i}$*? Should we compute also* $\nabla_{\mathbf{i}} \hat{R}$*?*
– Well! You may try! *But, obviously* i_k *is an integer!*

Optimal Segmentation

Optimal approach for finding $\mathbf{i}$ *is to train the NN for all possible choice for* $\mathbf{i}$ *and then find the final training loss* $\hat{R}(\mathbf{i})$*. The optimal segmentation is then given by*

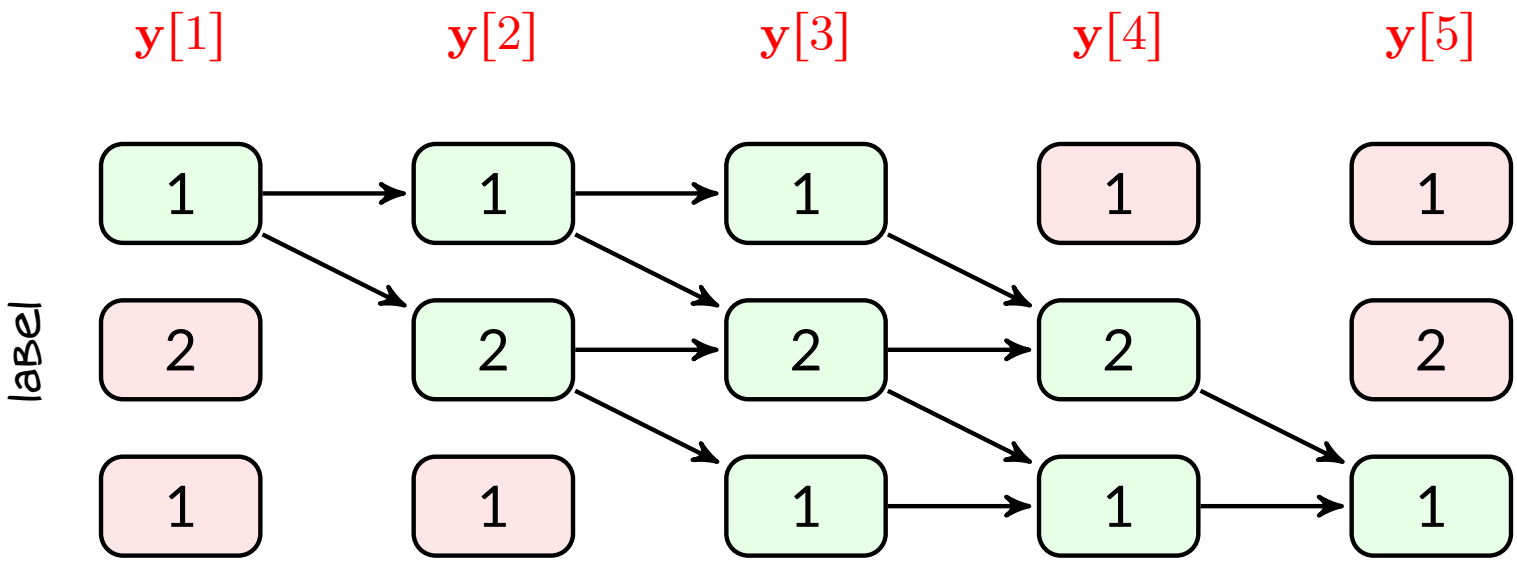
$$\mathbf{i}^{\star} = \underset{\mathbf{i}}{\operatorname{argmin}}\, \hat{R}(\mathbf{i})$$

+ *Is it computationally feasible?*
– No! *The number of possible choice for* $\mathbf{i}$ *grows exponentially with* T*! We need to go for sub-optimal approaches*
  ↳ *We use causality to develop an efficient algorithm* $\rightsquigarrow$ *CTC*

CTC: *Intuition*

We intend to compare each of $\mathbf{y}[1], \ldots, \mathbf{y}[5]$ *with a label*

- *We know that the label sequence is 121*
  - ↳ *First output is definitely in the first segment: it's label is definitely 1*
  - ↳ *Second output could be still in the first segment or in the second segment*
  - ↳ *Third output could be in the first, second, or third segment*
  - ↳ *Labels should finish: fourth output cannot be in first segment*
  - ↳ *Last output could be only in the third segment*



In PyTorch: *CTC Loss*

We can access CTC loss in `torch.nn` module as

```
torch.nn.CTCLoss()
```

Few notes about CTC loss implementation

- *To handle repetition in the sequence, we define a blank label*
  - ↳ *It is out of our set of classes*
  - ↳ *By default, it is set to* `blank = class 0`
- *When we define our model, we should take blank label into account*
  - ↳ *If we do classification with C classes, model should return $C + 1$ classes*
- *PyTorch considers cross-entropy loss function, i.e., $\mathcal{L}(\mathbf{y}, \tilde{\mathbf{v}}) = \mathrm{CE}(\mathbf{y}, \tilde{\mathbf{v}})$*

Check the Appendix for details on CTC